

DS	Session	Tree 1	Question Information	Level 1	Challenge 67
Question description The sam is enjoying a wonderful vacation in Byteland. What makes the sam impressed the most is the road system of the country. Byteland has N cities numbered 1 through N. City 1 is the capital of Byteland. The country also has $N-1$ bidirectional roads connecting the cities. The i-th road connects two different cities u_i and v_i. In this road system, people can travel between every pair of different cities by going through exactly one path of roads. The roads are arranged in such a way that people can distinguish two cities only when both cities have different number of roads connected to it. Such two cities will be considered similar. For example, city A is similar to the capital if the number of roads connected to city A is equal to the number of roads connected to the capital. On each day during the vacation, the sam wants to have a trip as follows. He chooses two cities A and B such that the sam will visit city B if he goes from A to the capital using the shortest path. Then, the sam will visit the cities on the shortest path from A to B through this path. Please note that A may be equal to B; that means the sam will enjoy the day in a single city. The sam does not want to have similar trips. Two trips are considered similar if and only if they both have the same number of visited cities and for each i, the i-th city visited in one trip is similar to the i-th city visited in the other trip. The sam wants to have as many different, namely not similar, trips as possible. Help him count the maximum number of possible trips such that no two of them are similar. Input Format The first line of the input contains a single integer N. The i-th line of next $N-1$ lines contains two space-separated integers u_i and v_i, denoting the i-th road. Output Format Output a single line containing the maximum number of different trips.					

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

3
2 1
3 1

EXPECTED OUTPUT

3

Test Case 2

INPUT (STDIN)

4
2 1
3 1
2 4

EXPECTED OUTPUT

5

▼ Mandatory Test Cases

Test Case 1

KEYWORD

struct state

Test Case 2

KEYWORD

const int MAXL=200005;

Test Case 3

KEYWORD

state st[MAXL]

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

1

Test Case 2

TOKEN COUNT

700

Test Case 3

NLOC

67

```
Tree17.c +
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAXL 200005
5 #define HASHSIZE (1 << 21)
6 #define HASHMASK (HASHSIZE - 1)
7
8 int deg[MAXL];
9 int head[MAXL], to[2 * MAXL], nxt[2 * MAXL], ecnt;
10 long long keys[HASHSIZE];
11 int vals[HASHSIZE];
12 int stack_node[MAXL];
13 int stack_parent[MAXL];
14 int stack_edge[MAXL];
15 int path[MAXL];
16
17 void add_edge(int a, int b) {
18     to[ecnt] = b;
19     nxt[ecnt] = head[a];
20     head[a] = ecnt++;
21 }
22
23 int get_child(int parent, int d) {
24     long long key = (long long)parent * 1000003LL + d;
25     int h = (unsigned)key & HASHMASK;
```

```

26 while (keys[h] != 0 && keys[h] != key) h = (h + 1) & HASHMASK;
27 if (keys[h] == 0) return -1;
28 return vals[h];
29}
30
31void set_child(int parent, int d, int child) {
32     long long key = (long long)parent * 1000003LL + d;
33     int h = (unsigned)key & HASHMASK;
34     while (keys[h] != 0 && keys[h] != key) h = (h + 1) & HASHMASK;
35     keys[h] = key;
36     vals[h] = child;
37}
38
39int main() {
40     int n, i;
41     if (scanf("%d", &n) != 1) return 0;
42     for (i = 0; i < HASHSIZE; i++) keys[i] = 0;

```

```

43  ecnt = 0;
44  for (i = 1; i <= n; i++) head[i] = -1;
45  for (i = 0; i < n - 1; i++) {
46      int u, v;
47      scanf("%d %d", &u, &v);
48      add_edge(u, v);
49      add_edge(v, u);
50  }
51  for (i = 1; i <= n; i++) {
52      int c = 0;
53      for (int e = head[i]; e != -1; e = nxt[e]) c++;
54      deg[i] = c;
55  }

```

main | CH.SC.U4CSE25225 | Tree17.c

Ln 107, Col 1 Spaces: 4 UTF-8 CRLF C

```

56
57  long long ans = 0;
58  int sp = 0;
59  int path_top = 0;

```

```
60  stack_node[sp] = 1;
61  stack_parent[sp] = 0;
62  stack_edge[sp] = head[1];
63  path[path_top++] = deg[1];
64  sp++;
65  int node = 0;
66  for (i = path_top - 1; i >= 0; i--) {
67      int nx = get_child(node, path[i]);
68      if (nx == -1) {
69          nx = (int)(ans + 1);
70          ans++;
71          set_child(node, path[i], nx);
72      }
73      node = nx;
74  }
75
```

```
76 while (sp > 0) {
77     int u = stack_node[sp - 1];
78     int par = stack_parent[sp - 1];
79     int e = stack_edge[sp - 1];
80     if (e == -1) {
81         sp--;
82         path_top--;
83         continue;
84     }
85     stack_edge[sp - 1] = nxt[e];
86     int v = to[e];
87     if (v == par) continue;
88     path[path_top++] = deg[v];
89     node = 0;
90     for (i = path_top - 1; i >= 0; i--) {
91         int nx = get_child(node, path[i]);
```

```
92         if (nx == -1) {
93             nx = (int)(ans + 1);
94             ans++;
95             set_child(node, path[i], nx);
96         }
97         node = nx;
98     }
99     stack_node[sp] = v;
100    stack_parent[sp] = u;
101    stack_edge[sp] = head[v];
102    sp++;
103 }
104
105 printf("%lld\n", ans);
106 return 0;
107 }
```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

3

2 1

3 1

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree17.exe

3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

4

2 1

3 1

2 4

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Tree17.exe

5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>